nscc
Information Technology
Generalist

PROG1400 – Study Guide

Course: PROG1400 – Introduction to Object-Oriented Programming **Instructor:** Davis Boudreau **Midterm Type:** Practical (Hands-On) **Learning Outcomes Assessed:**

- LO2 – Implement OOP design principles
- LO3 – Manage and distribute code through versioning
- LO4 – Develop solutions using UML documentation

1 What This Midterm Is Really Testing

This is **not** a memory test.

It evaluates whether you can:

- Model a system using UML
- Use professional tools (VS Code, GitHub, Mermaid)
- Apply OOP principles in Python
- Use Copilot intelligently
- Work independently and troubleshoot

It measures your **process, reasoning, and practical skill**.

2 What You Must Be Comfortable Doing

A. UML with Mermaid (LO4)

You must be able to:

✔ Install & Verify Mermaid in VS Code

- Install Mermaid preview extension
- Open a `.md` file
- Preview diagrams successfully

✔ Write Basic Mermaid Class Diagram

You should be comfortable writing:

```
classDiagram
    class Player {
        -position
        +move()
    }

    class Enemy {
        -position
        +move()
    }

    Player --> Enemy
```

You should understand:

- What a class is
- What attributes are
- What methods are
- What relationships mean

✔ Write Basic Mermaid Sequence Diagram

Example structure:

```
sequenceDiagram
    Player->>Game: move()
    Game->>Maze: checkCollision()
    Maze-->>Game: result
    Game-->>Player: updatePosition()
```

You must understand:

- Actors
- Messages
- Order of interaction

✔ Use Copilot Properly

You should be able to:

- Write clear prompts
- Modify Copilot output
- Explain what the diagram represents

Good example prompt:

> "Generate a Mermaid class diagram for a simple maze game with Player, Enemy, Maze, and GameController. Include attributes and methods."

Bad practice:

- Accepting the output without understanding it.

B. GitHub & Version Control (LO3)

You must be comfortable with:

✔ Repository Structure

- `src/`
- `docs/uml/`
- Clean organization

✔ Basic Git Workflow

- `git add`
- `git commit`
- `git push`

✔ Meaningful Commit Messages

Good:

```
Added initial class diagram for player and enemy
```

Bad:

```
update
```

✔ Explaining:

- Why version control matters
- Difference between local and remote
- Why small commits are better than one large commit

C. OOP Implementation in Python (LO2)

You should be able to:

✔ Write a Simple Class

```
class Player:
    def __init__(self, position):
        self._position = position

    def move(self):
        print("Player moves")
```

✔ Demonstrate Encapsulation

- Private attributes (`_position`)
- Controlled access via methods

✔ Demonstrate Inheritance

```
class Character:
    def move(self):
        pass

class Player(Character):
    def move(self):
        print("Player moves")
```

✔ Demonstrate Polymorphism

Multiple classes implementing `move()` differently.

---

3 What You Should Review Before the Midterm

Review Your Own Project

Be ready to:

- Explain your objects
- Explain responsibilities
- Explain your UML decisions

If you cannot describe your game clearly, review your:

- Project Brief
- Object Responsibilities Worksheet
- UML v1

Review These Concepts

Make sure you can explain:

- Encapsulation (without saying just “private”)
- Abstraction (focus on behavior)
- Inheritance (reducing duplication)
- Polymorphism (same method, different behavior)
- Aggregation (has-a relationship)
- Interfaces (what something can do)

---

4 Practice Checklist (Do This Before the Midterm)

Try this on your own:

- ☑ Create a new repo
- ☑ Add a Mermaid class diagram
- ☑ Modify it
- ☑ Commit changes twice
- ☑ Push to GitHub
- ☑ Write a simple Python class from the diagram
- ☑ Explain it out loud

If you can do that without panic, you are ready.

---

5 What the Instructor Is Watching For

You are being evaluated on:

- Time management
- Problem solving
- Resource usage
- Knowledge
- Hands-on skill

You are **not** being evaluated on:

- Fancy graphics
- Perfect syntax
- Large amounts of code

Clarity > complexity.

---

6 Common Mistakes to Avoid

❌ One giant commit at the end ❌ Letting Copilot do everything ❌ UML that doesn't match code ❌ One class doing everything ❌ Not being able to explain your design

---

7 How to Reduce Anxiety

During the midterm:

- Start with UML first
- Make small commits
- Work in stages
- If stuck, simplify
- Think in terms of responsibilities

You are demonstrating **applied understanding**, not perfection.

---

8 Final Self-Assessment Questions

Before the midterm, ask yourself:

- Can I create a class diagram from memory?
- Can I create a sequence diagram?
- Can I explain my UML decisions?
- Can I commit properly?
- Can I write a simple OOP class without Googling?

If yes — you are prepared.

Support:

- Please do not hesitate to **reach out to your instructor before the exam if you have any concerns.**
